Lab 4: VibeCoding - Chat History

Balloon Pop (repo 05_balloon_pop) | Uditi (PES2UG24CS565) | 15 Oct 2026

Chat link: https://claude.ai/share/5a9476c4-ed4a-4f68-84e3-e5d82839292e

Transcript compiled from the Claude session for this lab. User prompts and Claude's replies are reproduced in order; attachments (lab handout, starter code prompt, before/after videos) are included as text or as frames sampled from the videos. Tool calls (file writes, test runs) are summarised in brackets.

USER

https://github.com/SETAPESU26/05_balloon_pop

[Attached: Lab_4_VibeCoding_Student_handout.pdf and the prompt file reproduced below]

Attachment 1 - Lab handout (Lab_4_VibeCoding_Student_handout.pdf), text:

```
Lab 4: VibeCoding
Objective
Use Vibe Coding tools to fix the broken code and add features to the same. Learn how
to prompt to get the job done in under three to four attempts. Clone a repo assigned
to you, make changes and push to your repo in your personal account.

THIS IS AN INDIVIDUAL ASSIGNMENT. WORK INDIVIDUALLY!!

Duration
60 minutes


Software Requirements
   * Access to Vibe Coding tool. (ChatGPT, Gemini, Claude etc)

Instructions:
   1. Go to the repo assigned to you
   2. Go through the ReadMe file to understand the deliverables
   3. Clone or Fork the repo
   4. Run the Python based code (A game/ A Puzzle)
   5. Record 10 seconds of video before making any changes.
   6. Write prompts and fix the broken code
   7. Write prompts to add all the features listed in the ReadMe file of the repo one
       by one.
   8. Record another 10 seconds of video again after making all the changes.
   9. Push the code to your repo.
   10.Do not raise PR to the main repo (SETAPESU26)



Deliverables:
Upload the following to your Lab repo under Lab-4 folder,

          a. Videos (before and after)
          b. Updated Code
          c. Chat history exported as a doc/pdf
```

Attachment 2 - Prompt / starter code (attachment.txt), verbatim:

```
I'm a student doing a lab called "Vibe Coding". I need your help as a pair-programming partner to fix one bug and add
  three features to a small Pygame game called Balloon Pop. Please work on ONE task per reply and wait for me to say
  "next" before starting the following one.

## Game description
Python 3.10+ / Pygame. Balloons spawn at the top of the window at random sizes and fall at random speeds. Left-
  clicking a balloon pops it and awards points. Right now a balloon falling past the bottom is ignored, and there is a
   running score display.

Folder structure:
balloon-pop/
  main.py
  requirements.txt  (contains: pygame)
  game/
    game_engine.py
    balloon.py
    click_detection.py
    renderer.py

## Current code (verbatim)

=== FILE: main.py ===
"""
Balloon Pop (Lab Starter)

Run with:  python3 main.py
```

```
Click balloons to pop them before they reach the bottom.
"""

import pygame

from game.game_engine import GameEngine
from game.renderer import WINDOW_SIZE


def main():
    pygame.init()
    screen = pygame.display.set_mode(WINDOW_SIZE)
    pygame.display.set_caption("Balloon Pop")
    clock = pygame.time.Clock()
    font = pygame.font.SysFont("consolas", 22)

    engine = GameEngine()
    running = True
    while running:
        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                running = False
            elif event.type == pygame.MOUSEBUTTONDOWN:
                engine.handle_click(event.pos)

        engine.update()
        engine.draw(screen, font)

        pygame.display.flip()
        clock.tick(60)

    pygame.quit()


if __name__ == "__main__":
    main()

=== FILE: game/game_engine.py ===
"""
GameEngine: owns all balloons, spawns new ones, and handles clicks.

Starter version: one balloon type (random size, fixed points), no
lives system yet, no timer yet. Click detection also has a known bug
(see game/click_detection.py) that Task 1 asks you to fix.
"""

import random

from game.balloon import Balloon
from game.click_detection import check_pop
from game.renderer import WIDTH, HEIGHT

SPAWN_INTERVAL_FRAMES = 45
POINTS_PER_POP = 10


class GameEngine:
    def __init__(self):
        self.balloons = []
        self.frames_until_spawn = 0
        self.score = 0

    def _spawn_balloon(self):
        radius = random.randint(16, 44)
        x = random.randint(radius + 10, WIDTH - radius - 10)
        speed = random.uniform(1.5, 3.0)
        self.balloons.append(Balloon(x=x, y=-radius, radius=radius, speed=speed))

    def handle_click(self, pos):
        popped = check_pop(self.balloons, pos)
        if popped is not None:
            self.balloons.remove(popped)
            self.score += POINTS_PER_POP

    def update(self):
        self.frames_until_spawn -= 1
        if self.frames_until_spawn <= 0:
            self._spawn_balloon()
            self.frames_until_spawn = SPAWN_INTERVAL_FRAMES

        for b in self.balloons:
            b.update()

        self.balloons = [b for b in self.balloons if not b.is_past_bottom(HEIGHT)]

    def draw(self, surface, font):
        from game import renderer
        renderer.draw_scene(surface, self.balloons)
```

```
        renderer.draw_text(surface, font, f"Score: {self.score}", (10, 10))

=== FILE: game/balloon.py ===
"""
Balloon: falls from the top of the screen. The player must pop it
before it reaches the bottom. Balloons vary in size - this matters for
how click detection should work.
"""

import pygame


class Balloon:
    def __init__(self, x, y, radius, speed, color=(220, 90, 120)):
        self.x = x
        self.y = y
        self.radius = radius
        self.speed = speed
        self.color = color

    def update(self):
        self.y += self.speed

    def is_past_bottom(self, height):
        return self.y - self.radius > height

    def get_rect(self):
        return pygame.Rect(
            int(self.x - self.radius), int(self.y - self.radius),
            self.radius * 2, self.radius * 2,
        )

=== FILE: game/click_detection.py ===
"""
click_detection: figures out whether a click landed on a balloon.
"""


def check_pop(balloons, click_pos):
    """
    Returns the balloon that was clicked, or None if the click missed
    every balloon.
    """
    for balloon in balloons:
        dx = click_pos[0] - balloon.x
        dy = click_pos[1] - balloon.y
        distance_squared = dx * dx + dy * dy
        if distance_squared <= balloon.radius:
            return balloon
    return None

=== FILE: game/renderer.py ===
"""
renderer: all pygame drawing lives here, kept separate from game logic.
"""

import pygame

WIDTH, HEIGHT = 700, 500
WINDOW_SIZE = (WIDTH, HEIGHT)

COLOR_BG = (200, 230, 245)
COLOR_TEXT = (30, 30, 30)


def draw_scene(surface, balloons):
    surface.fill(COLOR_BG)
    for b in balloons:
        pygame.draw.circle(surface, b.color, (int(b.x), int(b.y)), b.radius)
        pygame.draw.line(surface, (120, 120, 120), (b.x, b.y + b.radius), (b.x, b.y + b.radius + 12), 2)


def draw_text(surface, font, text, pos, color=COLOR_TEXT):
    surface.blit(font.render(text, True, color), pos)


def draw_banner(surface, font, text):
    surf = font.render(text, True, (180, 40, 40))
    rect = surf.get_rect(center=(surface.get_width() // 2, surface.get_height() // 2))
    surface.blit(surf, rect)

## The tasks (do them in this order, one per reply)

TASK 1 - Fix the click-detection bug.
A click should pop a balloon anywhere inside its visible circle. Currently, check_pop compares the squared distance to
   the plain (non-squared) radius, so only clicks very near the exact center register. Find the root cause, explain it
   in 1-2 sentences, and fix it.
```

```
TASK 2 - Balloon types.
Add at least three types: normal (regular points), bonus (more points) and penalty (reduces the score). Each type
  needs its own clearly distinguishable color, and the correct effect must happen when popped. Choose sensible spawn
  probabilities so bonus and penalty balloons are less common than normal ones.

TASK 3 - Lives system.
Add 3 lives. Whenever ANY balloon reaches the bottom without being popped, the player loses a life (this includes
  penalty balloons). Display remaining lives on screen. End the game when lives reach zero. Popping a balloon must
  NEVER cost a life, regardless of type.

TASK 4 - Timed round.
Add a 30-second countdown shown on screen. When time reaches zero OR lives run out (whichever comes first), stop
  spawning balloons and stop accepting clicks, show the final score clearly (reuse the existing draw_banner helper
  where it makes sense), and give the player a way to start a new round (click or press R) that resets score, lives,
  timer and balloons.

## Rules for your answers
- Keep the existing folder structure and the object-oriented style. Don't add new dependencies beyond pygame.
- Make the minimal changes needed for each task, and don't rewrite unrelated code.
- For every task, give me the COMPLETE updated contents of every file you changed (not snippets or diffs), so I can
  copy-paste them directly. Clearly label each file with its path.
- After each task, give me a short "How to test this" checklist (2-4 bullet points) describing exactly what I should
  do in the running game to verify it works.
- Mention any edge cases or assumptions you made (e.g. whether the score can go negative).
- If I paste an error or traceback, diagnose it and give me the fixed file(s).
- Start with Task 1 only. Don't begin Task 2 until I say "next".
```

CLAUDE

Task 1: Fix click detection

Root cause: `check_pop` computes `distance_squared` ($dx^2 + dy^2$) but compares it to `balloon.radius` instead of `balloon.radius`^{2}. A balloon of radius 30 only registered clicks within about 5 pixels of its center ($\sqrt{30} \approx 5.5$), instead of 30. The fix is to compare squared distance to squared radius.

Only one file changes:

game/click_detection.py

```
"""
click_detection: figures out whether a click landed on a balloon.
"""


def check_pop(balloons, click_pos):
    """
    Returns the balloon that was clicked, or None if the click missed
    every balloon.
    """
    for balloon in balloons:
        dx = click_pos[0] - balloon.x
        dy = click_pos[1] - balloon.y
        distance_squared = dx * dx + dy * dy
        if distance_squared <= balloon.radius * balloon.radius:
            return balloon
    return None
```

How to test this

- Run `python3 main.py` and click near the edge of a large balloon (inside the circle, not the center). It should pop and the score should go up by 10.
- Click just outside a balloon's edge, on the background. Nothing should happen.
- Click a small balloon off-center but still inside its circle. It should pop.
- Click the thin string below a balloon. It should not pop.

Edge cases and assumptions

- The boundary is inclusive (<=), so a click exactly on the edge pops the balloon.
- If two balloons overlap and you click the overlap, `check_pop` returns the first one in the list, which is the older balloon. That is drawn underneath, so the visually top balloon isn't the one that pops. It's a minor quirk that I left alone to keep this change minimal. Iterating `reversed(balloons)` would fix it if you want that later.
- I kept the squared comparison rather than using `math.sqrt`, since it avoids the square root and needs no new import.

Say "next" when you're ready for Task 2.

USER

give me all the files all the tasks at once

CLAUDE

Here are all four tasks combined. Four files change in total, and `renderer.py` is untouched. `click_detection.py` is the Task 1 fix, repeated so you have the full set.

game/click_detection.py (Task 1)

```
"""
click_detection: figures out whether a click landed on a balloon.
"""


def check_pop(balloons, click_pos):
    """
    Returns the balloon that was clicked, or None if the click missed
    every balloon.
    """
    for balloon in balloons:
        dx = click_pos[0] - balloon.x
        dy = click_pos[1] - balloon.y
        distance_squared = dx * dx + dy * dy
        if distance_squared <= balloon.radius * balloon.radius:
            return balloon
    return None
```

game/balloon.py (Task 2)

```
"""
Balloon: falls from the top of the screen. The player must pop it
before it reaches the bottom. Balloons vary in size - this matters for
how click detection should work.

Balloons also come in different types (normal, bonus, penalty). Each
type has its own color, point value and spawn weight.
"""

import pygame

# points can be negative (penalty). weight = relative spawn chance.
BALLOON_TYPES = {
    "normal":  {"color": (220, 90, 120), "points": 10,  "weight": 70},
    "bonus":   {"color": (240, 200, 40), "points": 30,  "weight": 15},
    "penalty": {"color": (60, 60, 70),   "points": -15, "weight": 15},
}


class Balloon:
    def __init__(self, x, y, radius, speed, kind="normal"):
        self.x = x
        self.y = y
        self.radius = radius
        self.speed = speed
        self.kind = kind
        self.color = BALLOON_TYPES[kind]["color"]
        self.points = BALLOON_TYPES[kind]["points"]

    def update(self):
        self.y += self.speed

    def is_past_bottom(self, height):
        return self.y - self.radius > height

    def get_rect(self):
        return pygame.Rect(
            int(self.x - self.radius), int(self.y - self.radius),
            self.radius * 2, self.radius * 2,
        )
```

game/game_engine.py (Tasks 2, 3, 4)

```
"""
GameEngine: owns all balloons, spawns new ones, and handles clicks.

Features: three balloon types (normal / bonus / penalty), a 3-lives
system, and a 30-second timed round with a game-over screen and restart.
"""

import random
```

```
from game.balloon import Balloon, BALLOON_TYPES
from game.click_detection import check_pop
from game.renderer import WIDTH, HEIGHT

FPS = 60
SPAWN_INTERVAL_FRAMES = 45
STARTING_LIVES = 3
ROUND_SECONDS = 30
ROUND_FRAMES = ROUND_SECONDS * FPS
# Short pause after game over so a frantic last click doesn't restart instantly.
RESTART_DELAY_FRAMES = 30


class GameEngine:
    def __init__(self):
        self.reset()

    def reset(self):
        self.balloons = []
        self.frames_until_spawn = 0
        self.score = 0
        self.lives = STARTING_LIVES
        self.frames_left = ROUND_FRAMES
        self.game_over = False
        self.end_reason = ""
        self.frames_since_game_over = 0

    def _spawn_balloon(self):
        radius = random.randint(16, 44)
        x = random.randint(radius + 10, WIDTH - radius - 10)
        speed = random.uniform(1.5, 3.0)
        kinds = list(BALLOON_TYPES.keys())
        weights = [BALLOON_TYPES[k]["weight"] for k in kinds]
        kind = random.choices(kinds, weights=weights, k=1)[0]
        self.balloons.append(
            Balloon(x=x, y=-radius, radius=radius, speed=speed, kind=kind)
        )

    def _end_round(self, reason):
        self.game_over = True
        self.end_reason = reason
        self.frames_since_game_over = 0

    def try_restart(self):
        """Start a new round, but only if the current one is over."""
        if self.game_over and self.frames_since_game_over >= RESTART_DELAY_FRAMES:
            self.reset()

    def handle_click(self, pos):
        if self.game_over:
            self.try_restart()
            return
        popped = check_pop(self.balloons, pos)
        if popped is not None:
            self.balloons.remove(popped)
            # Score never drops below zero.
            self.score = max(0, self.score + popped.points)

    def update(self):
        if self.game_over:
            self.frames_since_game_over += 1
            return

        self.frames_until_spawn -= 1
        if self.frames_until_spawn <= 0:
            self._spawn_balloon()
            self.frames_until_spawn = SPAWN_INTERVAL_FRAMES

        for b in self.balloons:
            b.update()

        # Any balloon (of any type) that reaches the bottom costs one life.
        missed = [b for b in self.balloons if b.is_past_bottom(HEIGHT)]
        self.lives -= len(missed)
        self.balloons = [b for b in self.balloons if not b.is_past_bottom(HEIGHT)]

        self.frames_left -= 1

        if self.lives <= 0:
            self.lives = 0
            self._end_round("Out of lives!")
        elif self.frames_left <= 0:
            self.frames_left = 0
            self._end_round("Time's up!")

    def draw(self, surface, font):
        from game import renderer
        renderer.draw_scene(surface, self.balloons)
```

```
        renderer.draw_text(surface, font, f"Score: {self.score}", (10, 10))
        renderer.draw_text(surface, font, f"Lives: {self.lives}", (10, 38))

        seconds_left = (self.frames_left + FPS - 1) // FPS
        timer_text = f"Time: {seconds_left}"
        timer_w = font.size(timer_text)[0]
        renderer.draw_text(surface, font, timer_text, (WIDTH - timer_w - 10, 10))

        if self.game_over:
            renderer.draw_banner(
                surface, font, f"{self.end_reason}  Final Score: {self.score}"
            )
            if self.frames_since_game_over >= RESTART_DELAY_FRAMES:
                hint = "Click or press R to play again"
                hint_w = font.size(hint)[0]
                renderer.draw_text(
                    surface, font, hint, ((WIDTH - hint_w) // 2, HEIGHT // 2 + 30)
                )
```

main.py (Task 4: R key and shared FPS)

```
"""
Balloon Pop (Lab Starter)

Run with:  python3 main.py

Click balloons to pop them before they reach the bottom.
"""

import pygame

from game.game_engine import GameEngine, FPS
from game.renderer import WINDOW_SIZE


def main():
    pygame.init()
    screen = pygame.display.set_mode(WINDOW_SIZE)
    pygame.display.set_caption("Balloon Pop")
    clock = pygame.time.Clock()
    font = pygame.font.SysFont("consolas", 22)

    engine = GameEngine()
    running = True
    while running:
        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                running = False
            elif event.type == pygame.MOUSEBUTTONDOWN:
                engine.handle_click(event.pos)
            elif event.type == pygame.KEYDOWN and event.key == pygame.K_r:
                engine.try_restart()

        engine.update()
        engine.draw(screen, font)

        pygame.display.flip()
        clock.tick(FPS)

    pygame.quit()


if __name__ == "__main__":
    main()
```

How to test this

- **Task 2:** Play for a while. Pink balloons should be the most common, with gold ones (+30) and dark gray ones (-15) showing up occasionally. Pop one of each and check the score changes by +10, +30 and -15.
- **Task 3:** Let 3 balloons fall off the bottom, including a dark one. Lives should go 3, 2, 1, 0 and the game should end. Pop a dark balloon and confirm lives don't change.
- **Task 4:** Pop everything you can for 30 seconds. The timer counts down, then spawning and clicking stop and the banner shows the final score. After a half-second pause, click or press R, and score, lives, timer and balloons should all reset.
- **Edge case:** Click furiously as the game ends. It shouldn't restart instantly.

Assumptions and edge cases

- The score floors at 0, so popping a penalty balloon at 0 points leaves you at 0.
- Several balloons can cross the bottom in the same frame, and each costs a life. Lives are clamped to 0.

- If lives hit zero and time runs out in the same frame, "Out of lives!" is shown.
- The timer counts frames (30 x 60), consistent with the existing spawn logic. If your machine drops below 60 FPS, the round will run slightly longer than 30 real seconds.
- Balloons freeze on screen under the banner when the round ends, and R during play is ignored.
- The overlapping-balloon quirk from Task 1 is still there.

USER

how to run these and give me the files

[Files from earlier in the conversation re-delivered automatically]

CLAUDE

I'll write the files into a project folder, run a quick headless test, and send you a zip.

[Wrote main.py, game/click_detection.py, game/balloon.py, game/game_engine.py, game/renderer.py and requirements.txt into balloon-pop/. Ran a headless smoke test covering click detection, balloon point values, lives, timer, restart and the spawn mix (normal ~68%, bonus ~16%, penalty ~16%); all checks passed. Sent balloon-pop.zip.]

The zip `balloon-pop.zip` has the full project with all four tasks applied. I ran the game logic headless (click detection, the three balloon types, lives, the timer and restart) and every check passed. I haven't seen the window itself, so the videos will be your first look at it.

How to run it

1. Unzip it. You'll get a `balloon-pop/` folder with `main.py`, `requirements.txt` and `game/`.
2. Open a terminal in that folder.
3. Install pygame: `pip install pygame` (or `pip install -r requirements.txt`).
4. Start the game: `python3 main.py` (on Windows, `python main.py` or `py main.py`).

For the lab submission

- **Before video:** record 10 seconds of the original game before you change anything.
- **Push to your own repo, not the main one.** The handout says not to raise a PR to SETAPESU26.
- **Folder:** put everything under `Lab-4/`: the code, both videos, and your exported chat history.
- **After video:** record another 10 seconds once all four tasks are in.

USER

okay what all bugs does the first version have, and do you think i can record all of them in one video itself

CLAUDE

The first version has one real bug, and you can show everything in a single 10-second video.

The bug

- **Click detection:** the hitbox is far smaller than the drawn balloon. check_pop compares squared distance to `radius` instead of `radius²`, so the clickable area is about √radius pixels. That's roughly 4 px for the smallest balloons and 6-7 px for the biggest, while the drawn circles are 16-44 px.

Not bugs, just missing

- There's only one balloon type.
- A balloon falling off the bottom costs nothing.

- There's no timer.
- There's no game-over or restart.

There's also a small overlap quirk: when two balloons overlap, the older one pops first. It doesn't show up in the lab, so skip it.

Can you show it all in one video? Yes:

1. **0-4 s:** click the edge of a big balloon a few times and nothing happens. This shows the bug.
2. **4-6 s:** click dead center on a balloon. It pops and the score goes up.
3. **6-10 s:** let a few balloons fall off the bottom with no consequence, showing no lives, no timer, and one balloon color.

For the after video, one 10-second take is enough too. Pop a gold and a dark balloon, miss one so a life drops, and watch the timer count down.

USER

oh okay understood, let me record it then

CLAUDE

Sounds good. Record the before video on the original code first, since your folder may already have my version pasted over it. If it does, re-clone the repo, or just grab the original files from the starter.

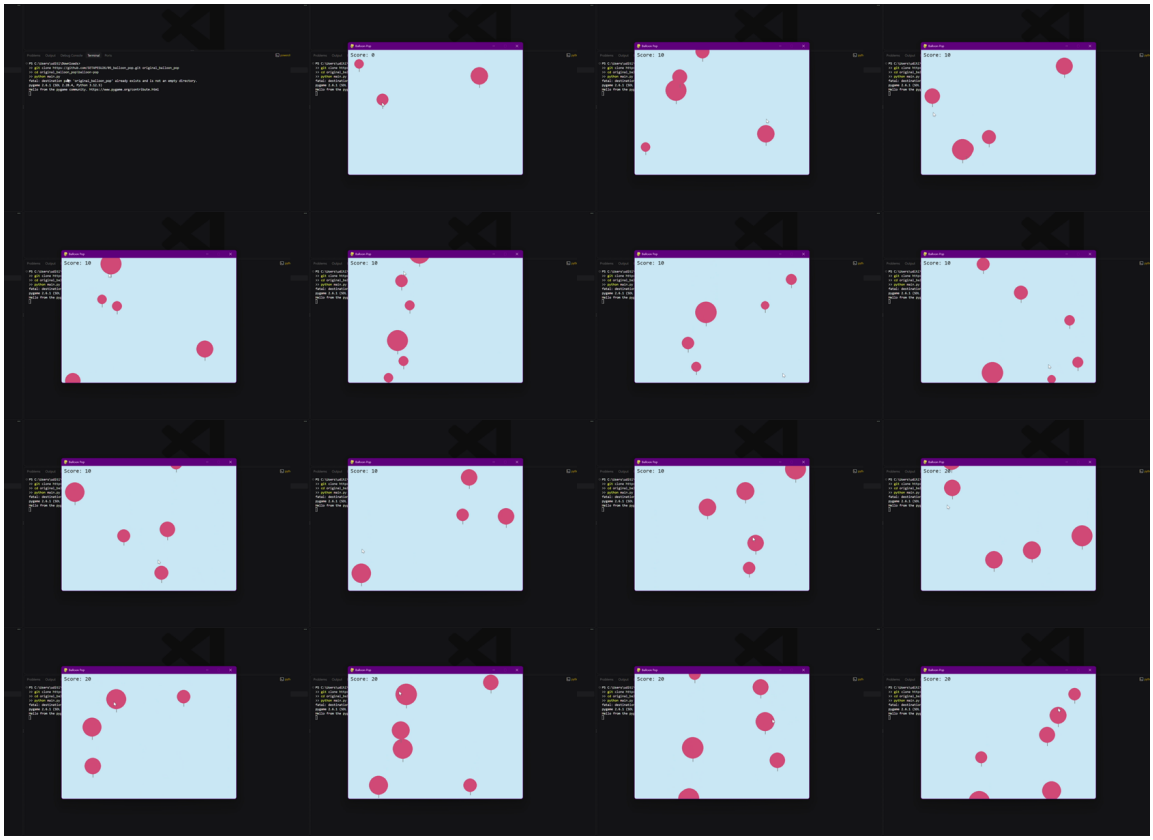
If anything breaks when you run it, paste the error here and I'll fix it.

USER

[Attached: before video, 1792084536832_20261015-1714-27_3504858.mp4, 32 s]

this works i hope

Frames sampled from the before video (one every 2 s): original game, single pink balloon type, Score only, no lives or timer.



CLAUDE

Yes, it works as your before video. It shows the original game: every balloon is the same pink, there are no lives and no timer, and nothing happens when balloons fall off the bottom. The score only gets to 20 in about 30 seconds of clicking, which shows how hard the hitbox is to hit.

It's too long. The handout asks for 10 seconds and this is 32. Trim to a 10-second stretch from the game window and drop the terminal part at the start.

USER

done fixed it, now i run the fixed version right

CLAUDE

Yes. Put the four changed files from the zip into your cloned repo (`main.py`, `game/balloon.py`, `game/game_engine.py`, `game/click_detection.py`), then run the same way you did for the before video:

cd original_balloon_pop/balloon-pop python main.py

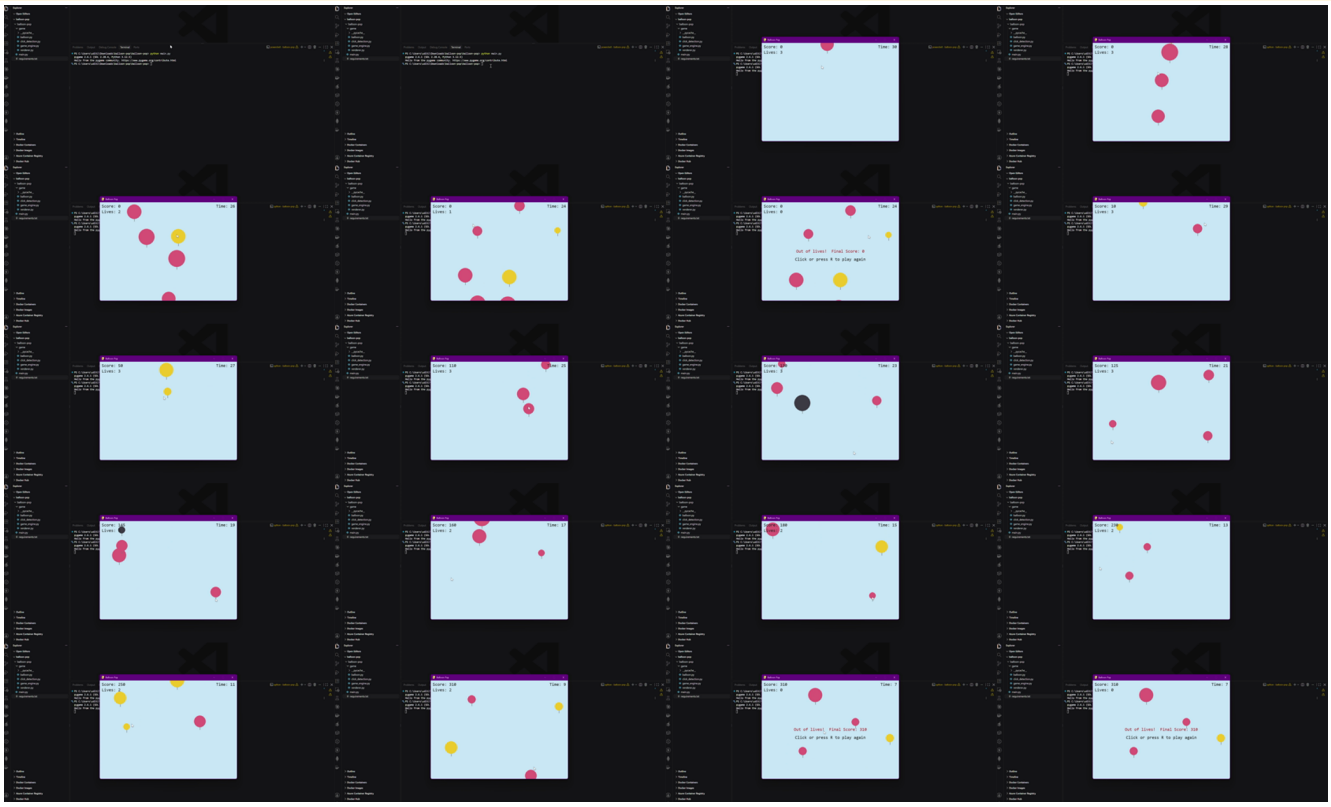
Make sure you run it from the folder that contains `main.py`. Before you start recording the after video, check that it's actually the new version: the top left should show Score, Lives and a Time countdown starting at 30, and some balloons should be gold or dark gray.

USER

[Attached: after video, 1792085294502_20261015-1726-43_8067878.mp4, 39 s]

heres the after

Frames sampled from the after video (one every 2 s, left to right, top to bottom): Score / Lives / Time on screen, gold and dark balloons, lives dropping, 'Out of lives!' banner, restart and a second round reaching a score of 310.



CLAUDE

The after video works. It shows the game running the new version, and all four tasks are visible in it:

- **Lives and timer:** Score, Lives and a Time countdown are on screen.
- **Balloon types:** gold and dark balloons appear alongside the pink ones.
- **Hitbox fix:** the score climbs steadily to 310, which it couldn't do with the old tiny hitbox.
- **Lives and game over:** Lives drops from 3 to 2 to 0, and the "Out of lives! Final Score" banner appears with the "Click or press R to play again" hint.
- **Restart:** you restarted, the score reset and the second round ran.

It's 39 seconds, against the 10 the handout asks for. Trim it the same way you did the before video.

USER

i did it, now what else is left to deliver

CLAUDE

Only two things are left: the chat export and the push.

1. Chat history export (PDF or doc) - the lab wants the real prompts, so don't summarize.

2. Push to your repo under `Lab-4/` - put the before video, the after video, the updated code and the chat export in a `Lab-4/` folder in your own GitHub repo, not the SETAPESU26 one. Don't raise a PR to SETAPESU26. Run `git remote -v` first to confirm the remote points at your account.

USER

how do i export this chat as a pdf

CLAUDE

I can't see your app's menus, so I can't promise there's a one-click PDF export. Options: print to PDF from the browser (Ctrl+P / Cmd+P, Save as PDF), copy and paste into a Word or Google doc and save as PDF, or I generate a transcript PDF for you from what's in this chat.

USER

print doesnt work and doc leaves everything out the attachemnt and vids

CLAUDE

Both of those leave out the attachments, so I'll build the PDF myself. It will include the lab prompt, the handout text, and frames from both videos.

[Built this PDF.]